

EDS Lab Assignments

Practical 2

Name: Sakshi Shivaji Mulay

PRN: 202501040061

3.1.1 Numpy Array Operations

Problem Statement:

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size` Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Code:

```
import numpy as np
rows, cols = map(int, input().split())
elements = []

for _ in range(rows):
    elements.extend(map(int, input().split()))

array = np.array(elements).reshape(rows, cols)
print(array)
print(array.ndim)
print(array.shape)
print(array.size)
```

Output:

CODETANTRA

HomeLearn Anywhere

202501040061@mitaoe.ac.inSupportLogout

3.1.1. Numpy Array Operations09:01

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using `ndim`
- The shape of the array using `shape`
- The total number of elements in the array using `size`

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
- The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

numpyarr...

```
1 import numpy as np
2
3 rows, cols = map(int, input().split())
4
5 elements = []
```

Average time0.033 s32.58 ms

Maximum time0.050 s50.00 ms

2 out of 2 shown test case(s) passed

10 out of 10 hidden test case(s) passed

Test case 137 msDebug

Expected output

Actual output

3 4

3 4

1 2 3 4

1 2 3 4

5 6 7 8

5 6 7 8

9 10 11 12

9 10 11 12

[[-1 -2 -3 -4]]

[[-1 -2 -3 -4]]

- [-5 -6 -7 -8]]

- [-5 -6 -7 -8]]

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.1 Numpy: Matrix Operations

Problem Statement:

The given code takes two 3×3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. Addition (matrix_a + matrix_b)
2. Subtraction (matrix_a – matrix_b)
3. Element-wise Multiplication (matrix_a * matrix_b)
4. Matrix Multiplication (matrix_a · matrix_b)
5. Transpose of Matrix A

Input Format:

- The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Code:

```
import numpy as np
```

```
# Input matrices print("Enter  
Matrix A:")
```

```
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
```

```
print("Enter Matrix B:")
```

```
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
```

```
# Addition
```

```
addition_result = matrix_a + matrix_b print("Addition  
(A + B):")
```

```
print(addition_result)
```

```
# Subtraction
```

```
subtraction_result = matrix_a - matrix_b
```

```
print("Subtraction (A - B):")
```

```
print(subtraction_result)
```

```
# Multiplication (element-wise)
```

```
elementwise_multiplication_result = matrix_a * matrix_b
```

```
print("Element-wise Multiplication (A * B):") print(elementwise_multiplication_result)
```

```
# Matrix multiplication (dot product)
```

```
matrix_multiplication_result = np.dot(matrix_a, matrix_b) print("A  
dot B:")
```

```
print(matrix_multiplication_result)
```

```
# Transpose transpose_a
```

```
= matrix_a.T
```

```
print("Transpose of A:")
```

```
print(transpose_a)
```

Output:

The screenshot displays the CODETANTRA online IDE interface. On the left, a task description for '3.2.1. Numpy: Matrix Operations' is shown, detailing the requirements for computing and displaying results of various matrix operations. The main editor area contains a Python script that implements these operations using NumPy. The script prompts the user to enter two 3x3 matrices, A and B, and then performs addition, subtraction, element-wise multiplication, matrix multiplication (dot product), and transposition of matrix A. The right sidebar shows the test results, indicating that all test cases passed. The bottom of the interface includes a terminal, test cases section, and navigation buttons.

Task:
You are required to compute and display the results of the following matrix operations:

1. Addition ($\text{matrix_a} + \text{matrix_b}$)
2. Subtraction ($\text{matrix_a} - \text{matrix_b}$)
3. Element-wise Multiplication ($\text{matrix_a} * \text{matrix_b}$)
4. Matrix Multiplication ($\text{matrix_a} \cdot \text{matrix_b}$)
5. Transpose of Matrix A

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

Test Results:

- Average time: 0.060 s (59.75 ms)
- Maximum time: 0.063 s (63.00 ms)
- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test case 1 (65 ms):

Expected output	Actual output
Enter Matrix A:	Enter Matrix A:
1 2 3	1 2 3
4 5 6	4 5 6
7 8 9	7 8 9
Enter Matrix B:	Enter Matrix B:
1 1 1	1 1 1

3.2.2 Numpy: Horizontal and Vertical Stacking of Arrays

Problem Statement:

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking: Stack the two matrices horizontally (side by side).
- Vertical Stacking: Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Code:

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
a = np.hstack((arr1, arr2))
print("Horizontal Stack:")
print(a)

# Perform vertical stacking (vstack)
b = np.vstack((arr1, arr2))
print("Vertical Stack:")
print(b)
```

Output:

CODETANTRA

HomeLearn Anywhere

202501040061@mitaoe.ac.inSupportLogout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays00.50

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
```

Average time0.056 s56.00 ms

Maximum time0.060 s60.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 160 ms

Debug

Expected output

Actual output

Enter-Array1:
1 2 3
4 5 6
7 8 9
Enter-Array2:
4 5 6

Enter-Array1:
1 2 3
4 5 6
7 8 9
Enter-Array2:
4 5 6

Terminal

Test cases

< Prev

Reset

Submit

Next >

3.2.3 Numpy: Custom Sequence Generation

Problem Statement:

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Code:

```
import numpy as np
# Take user input for the start, stop, and step of the sequence
start = int(input()) stop = int(input()) step = int(input())
# Generate the sequence using np.arange()
a = np.arange(start, stop, step) #
Print the generated sequence
print(a)
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '3.2.3. Numpy: Custom Sequence Generation' is shown, including the input/output formats and a sample test case. The main editor shows the Python code for generating a sequence using numpy. The right sidebar displays the test results, indicating that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The test cases show the expected output [3, 5, 7, 9] and the actual output [3, 5, 7, 9].

CodeTANTRA Home Learn Anywhere 202501040061@mitaoe.ac.in Support Logout

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the
4 start = int(input())
```

Average time 0.029 s 28.50 ms Maximum time 0.034 s 34.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 34 ms Debug

Expected output	Actual output
3	3
18	18
2	2
[3, 5, 7, 9]	[3, 5, 7, 9]

Test case 2 77 ms

Terminal Test cases

< Prev Reset Submit Next >

3.2.4 Numpy: Arithmetic and Statistical Operations, Mathematical Operations and Bitwise Operators

Problem Statement:

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Code:

```
import numpy as np
```

```
def array_operations(A, B):
```

```
    # Convert A and B to NumPy arrays
```

```
    A = np.array(A)
```

```
    B = np.array(B)
```

```
    # Arithmetic Operations
```

```
    sum_result = A + B    di_result
```

```
    = A - B
```

```
    prod_result = A * B
```

```
    # Statistical Operations
```

```
    mean_A = np.mean(A)    median_A
```

```
    = np.median(A)    std_dev_A =
```

```
    np.std(A)
```



```

# Bitwise Operations
and_result = A & B    or_result = A |
B

xor_result = A ^ B

# Output results with one space between each element    print("Element-
wise Sum:", ' '.join(map(str, sum_result)))    print("Element-wise Dierence:", '
'.join(map(str, di    _result)))    print("Element-wise Product:", '
'.join(map(str, prod_result)))

print(f"Mean of A: {mean_A}")
print(f"Median of A: {median_A}")    print(f"Standard
Deviation of A: {std_dev_A}")

print("Bitwise AND:", ' '.join(map(str, and_result)))
print("Bitwise OR:", ' '.join(map(str, or_result)))    print("Bitwise
XOR:", ' '.join(map(str, xor_result)))

A = list(map(int, input().split())) # Elements of array A
B = list(map(int, input().split())) # Elements of array B
array_operations(A, B)

```

Output:

The screenshot shows a coding platform interface with a problem statement on the left and test results on the right.

Problem Statement:

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematic...

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

- Arithmetic Operations:**
 - Compute the element-wise sum, difference, and product of the two arrays.
- Statistical Operations:**
 - Calculate the mean, median, and standard deviation of array A.
- Bitwise Operations:**
 - Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A_1 OR B_1).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

The right panel shows the test results for the function `array_operations`. It includes a code editor with the following code:

```

import numpy as np

def array_operations(A, B):
    # Convert A and B to NumPy arrays

```

The test results show that 1 out of 1 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The test case details are as follows:

Test Case	Expected Output	Actual Output
Test case 1	<pre> 1 2 3 4 5 6 7 8 Element-wise Sum: 6 8 10 12 Element-wise Difference: -4 -4 -4 -4 Element-wise Product: 5 12 21 32 Mean of A: 3.5 </pre>	<pre> 1 2 3 4 5 6 7 8 Element-wise Sum: 6 8 10 12 Element-wise Difference: -4 -4 -4 -4 Element-wise Product: 5 12 21 32 Mean of A: 3.5 </pre>

3.2.5 Numpy: Copying and Viewing Arrays

Problem Statement:

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:
Original array after modifying view: <original_array>
View array: <view_array>
- After modifying the copy:
Original array after modifying copy: <original_array> Copy
array: <copy_array>

Code:

```
import numpy as np
inputlist = list(map(int,input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

Output:

[Home](#)
[Learn Anywhere](#)

202501040061@mitaoe.ac.in
[Support](#)
[Logout](#)

3.2.5. Numpy: Copying and Viewing Arrays
08:45

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the `original_array` and assigning it to `view_array`.
- Creating a copy of the `original_array` and assigning it to `copy_array`.

After completing these steps, observe how modifying the view affects the `original_array`, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

```
Original array after modifying view: <original_array>
View array: <view_array>
```

- After modifying the copy:

```
Original array after modifying copy: <original_array>
```

copyAnd...
Submit

```

1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array

```

Average time
0.014 s
14.25 ms

Maximum time
0.015 s
15.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1 14 ms Debug

Expected output

Actual output

10 20 30 40 50 60 70 80

10 20 30 40 50 60 70 80

Original array after modifying view: [99 20 30 40 50 60 70 80]

Original array after modifying view: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

View array: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Original array after modifying copy: [99 20 30 40 50 60 70 80]

Terminal
Test cases

Prev

Reset

Submit

Next

3.2.6 Searching, Sorting, Counting, Broadcasting

Problem Statement:

The given code in the editor takes a single array, `array1`, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

1. Searching: Find the indices where search_value appears in array1 and print these indices.
2. Counting: Count how many times count_value appears in array1 and print the count.
3. Broadcasting: Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. Sorting: Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

Code:

```
import numpy as np
```

```
# Input array from the user
```

```
array1 = np.array(list(map(int, input().split())))
```

```
# Searching
```

```
search_value = int(input("Value to search: ")) count_value
```

```
= int(input("Value to count: ")) broadcast_value =
```

```
int(input("Value to add: "))
```

```
# Find indices where value matches in array1 a
```

```
= np.where(array1 == search_value)[0] print(a)
```

```
# Count occurrences in array1 b =
```

```
np.count_nonzero(array1 == count_value)
```

```
print(b)
```

```
# Broadcasting addition c =
array1 + broadcast_value
print(c)
```

```
# Sort the first array d
= np.sort(array1)
print(d)
```

Output:

The screenshot shows the CodeTANTRA online IDE interface. The code editor displays the following Python code:

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
```

The IDE provides instructions for the task:

The given code in the editor takes a single array, array1, as space-separated integers as input from the user. Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- Searching:** Find the indices where search_value appears in array1 and print these indices.
- Counting:** Count how many times count_value appears in array1 and print the count.
- Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

- A single line containing space-separated integers representing array1.
- An integer search_value represents the value to search for in the array.

The test cases panel shows the following results:

Test Case	Expected Output	Actual Output	Status
Test case 1	1 1 1 2 2 2	1 1 1 2 2 2	Passed
Value to search:	1	1	
Value to count:	2	2	
Value to add:	2	2	
Input	[0 1 2]	[0 1 2]	

The IDE also shows performance metrics: Average time 0.040 s, Maximum time 0.044 s. The test results indicate that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed.

3.2.7 Student Data Analysis and Operations

Problem Statement:

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
- Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
- Find the count of students who scored ≥ 90 in each subject: Count the number of students who scored 90 or more marks in each subject.
- Find the count of subjects in which each student scored ≥ 90 : Determine how many subjects each student scored 90 or more marks in.
- Print Subject 1 marks in ascending order: Sort and print the marks of students in Subject 1 in ascending order.
- Print students who scored between 50 and 90 in Subject 1: Display students who scored marks between 50 and 90 in Subject 1.
- Find index positions of students who scored 79 in Subject 1: Identify the index positions of students who scored exactly 79 marks in Subject 1.

Code:

```
import numpy as np
```

```
a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
```

```

roll = a[:, 0].astype(float)
s1 = a[:, 1] s2 = a[:, 2] s3
= a[:, 3]

# 1. Print all student details print("All
student Details:\n", a)

# 2. print total students print("Total
Students:", len(a))

# 3. Print all student Roll numbers
print("All Student Roll Nos", roll)

# 4. Print subject 1 marks
print("Subject 1 Marks", s1)

# 5. print minimum marks of Subject 2
print("Min marks in Subject 2", np.min(s2))

# 6. print maximum marks of Subject 3
print("Max marks in Subject 3", np.max(s3))

# 7. Print All subject marks
print("All subject marks:", a[:, 1:])

# 8. print Total marks of students
total = s1 + s2 + s3 print("Total
Marks", total)

# 9. print average marks of each student
print(np.round(total/3, 1))

# 10. print average marks of each subject
print("Average Marks of each subject", np.mean(a[:, 1:], axis = 0))

# 11. print average marks of S1 and S2
print("Average Marks of S1 and S2", np.mean([s1, s2], axis = 1))

# 12. print average marks of S1 and S3
print("Average Marks of S1 and S3", np.mean([s1, s3], axis = 1))

# 13. print Roll number who got maximum marks in Subject 3
print("Roll no who got maximum marks in Subject 3", float(roll[np.argmax(s3)]))

# 14. print Roll number who got minimum marks in Subject 2

```

```

print("Roll no who got minimum marks in Subject 2", float(roll[np.argmin(s2)]))

# 15. print Roll number who got 24 marks in Subject 2
print("Roll no who got 24 marks in Subject 2",roll[s2 == 24].astype(float).reshape(1, -1))

# 16. print count of students who got marks in Subject 1 < 40
print("Count of students who got marks in Subject 1 < 40", np.sum(s1 < 40))

# 17. print count of students who got marks in Subject 2 > 90
print("Count of students who got marks in Subject 2 > 90:", np.sum(s2 > 90))

# 18. print count of students in each subject who got marks >= 90
print("Count of students in each subject who got marks >= 90:", np.sum(a[:, 1:] >= 90, axis
= 0))

# 19. print count of subjects in which each student got marks >= 90 print("Roll
no:", roll)
print("Count of subjects in which student got marks >= 90:", np.sum(a[:, 1:] >= 90, axis =
1))

# 20. Print S1 marks in ascending order print(np.sort(s1))

# 21. Print S1 marks >= 50 and <= 90 print(a[(s1
>= 50) & (s1 <= 90)].astype(float))
print(a.astype(float))

# 22. Print the index position of marks 79 print(np.where(s1
== 79))

```

Output:

CODETANTRA

HomeLearn Anywhere

202501040061@mitaoe.ac.inSupportLogout

3.2.7. Student Data Analysis and Operations

00:52

Submit

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.

Find total students: Determine the total number of students in the dataset.

Print all student roll numbers: Extract and print the roll numbers of all students.

Print Subject 1 marks: Extract and print the marks of all students in Subject 1.

Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.

Find maximum marks in Subject 3: Identify the highest marks in Subject 3.

Print all subject marks: Display the marks of all students for each subject.

Find total marks of students: Compute the total marks for each student across all subjects.

Find the average marks of each student: Compute the average marks for each student.

Find average marks of each subject: Compute the average marks for all students in each subject.

Operation...

Submit

Debugger

1import numpy as np

2

3a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

4

5# 1. Print all student details

Average time0.008 s8.00 ms

Maximum time0.008 s8.00 ms

1 out of 1 shown test case(s) passed

Test case 16 ms

Debug

Expected output

Actual output

All student Details:

[[301., 67., 77., 88.]]

[[301., 67., 77., 88.]]

[302., 78., 88., 77.]

[302., 78., 88., 77.]

[303., 45., 56., 89.]

[303., 45., 56., 89.]

[304., 88., 98., 45.]

[304., 88., 98., 45.]

[305., 78., 88., 99.]

[305., 78., 88., 99.]

[[306., 60., 70., 56.]]

[[306., 60., 70., 56.]]

Terminal

Test cases

< Prev

Reset

Submit

Next >